Automate Everything: CI/CD, Cron, Webhooks & Backups

2026-04-25 · kind watermelon Elch

Manual deploys are technical debt. Automate or regret.

CI/CD: What It Is and Why It Matters

Continuous Integration means every push is automatically linted and tested. Broken code is caught before it reaches the server.

Continuous Deployment means every merge to `main` automatically deploys to the server, with no manual SSH-and-pull.

The pipeline: lint → test → deploy. Each step gates the next. If linting fails, tests do not run. If tests fail, the deploy does not happen.

GitHub Actions

GitHub Actions run workflows in response to repository events. The configuration lives in YAML files under `.github/workflows/`.

GitHub Actions: https://docs.github.com/en/actions
Workflow syntax: https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          python -m venv .venv
          source .venv/bin/activate
          pip install -r requirements.txt
          pip install ruff pytest

      - name: Lint
        run: ruff check .

      - name: Test
        run: pytest tests/
```

Key concepts: *triggers* (`push`, `pull_request`) define when a workflow runs. *Jobs* run on *runners*, GitHub-provided VMs. *Steps* are sequential commands within a job.

Using Secrets in Actions

Never hardcode secrets in workflow files. Store them in GitHub's repository settings under Settings → Secrets and variables → Actions. Reference them as `${{ secrets.MY_SECRET }}`. Never echo them in logs.

The Deploy Workflow

```
# .github/workflows/deploy.yml
```

```
name: Deploy
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to server
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SERVER_IP }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SSH_PRIVATE_KEY }}
          script: |
            cd /opt/pixelwise
            bash scripts/deploy.sh
```

Tests: Where They Live

The CI pipeline runs `pytest`, but writing tests is an entire discipline covered in dedicated courses. Here we show where tests live (`tests/`), what a test looks like, and how the CI pipeline runs them. A few example tests are provided in the boilerplate:

pytest: https://docs.pytest.org/
ruff (linter): https://docs.astral.sh/ruff/

```
# tests/test_classifier.py
from app.classifier import classify_batch
import numpy as np

def test_classify_batch_shape():
    images = np.zeros((2, 28, 28), dtype=np.uint8)
    results = classify_batch(images)
    assert len(results) == 2
    assert "prediction" in results[0]
    assert "confidence" in results[0]
```

The goal is to see the feedback loop: push code → tests run → green or red.

Exercise: CI Workflow

WRITE THE CI WORKFLOW. Create `.github/workflows/ci.yml` that on every push lints with `ruff` and runs `pytest` against the boilerplate tests in `tests/`:

EXERCISES are for practice and reinforcing concepts. If your VM is broken, restore the `B7-complete` snapshot. At the end of this session, take a snapshot named `B8-complete`, your safety net for the next block.

```
mkdir -p .github/workflows
nano .github/workflows/ci.yml
```

Use the YAML from the theory above. Push and watch the workflow run on GitHub.

The Deploy Script

A STANDALONE DEPLOY SCRIPT is reusable from both Actions and manual deploys:

```
#!/bin/bash
# scripts/deploy.sh
set -euo pipefail

cd /opt/pixelwise
git pull origin main
source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head
sudo systemctl restart pixelwise
echo "Deploy complete: $(date)"
```

Exercise: Deploy Workflow

WRITE THE DEPLOY WORKFLOW. Create `.github/workflows/deploy.yml` that on merge to `main`, SSHs to the server and runs the deploy script. Add your SSH key and server IP as GitHub Actions secrets.

Exercise: Deploy Script

WRITE THE DEPLOY SCRIPT. Create `scripts/deploy.sh` with the steps from the theory above: pull, install deps, run migrations, restart service.

```
mkdir -p scripts
nano scripts/deploy.sh
chmod +x scripts/deploy.sh
```

Cron: Scheduled Automation

CRON RUNS COMMANDS ON A SCHEDULE, no human required. The syntax has five fields:

```
# +-------- minute (0-59)
# | +------ hour (0-23)
# | | +---- day of month (1-31)
# | | | +-- month (1-12)
# | | | | + day of week (0-7, Sun=0 or 7)
# | | | | |
  * * * * *  command
```

COMMON PATTERNS:

```
0 2 * * *    # daily at 2:00 AM
*/5 * * * *  # every 5 minutes
0 0 * * 0    # weekly, Sunday midnight
```

Edit with `crontab -e`. List with `crontab -l`.

STUDENTS HAVE ALREADY SEEN AUTOMATION: `systemd` restarts their crashed service automatically (Block 5). `journalctl` rotates its own logs. Package managers run unattended security updates. Cron jobs are the explicit, user-controlled version of automation that is already everywhere under the hood.

Real-world cron uses: database backups, clearing temporary files, health checks that ping your service and alert on failure, certificate renewal, triggering retraining pipelines, sending summary reports. The common thread: anything that needs to happen reliably on a schedule without a human remembering.

Exercise: Nightly Backup Cron

SET UP THE NIGHTLY BACKUP CRON. On the server, create a cron job that runs nightly at 2:00 AM. `pg_dump` the database, then `rsync` the dump to the dev VM:

```
crontab -e
```

Add:

```
0 2 * * * pg_dump -U pixelwise pixelwise \
    > /tmp/pixelwise_$(date +\%Y\%m\%d).sql \
    && rsync -avz \
    /tmp/pixelwise_$(date +\%Y\%m\%d).sql \
    user@192.168.56.10:/backups/pixelwise/
```

On the dev VM, create the backup directory:

```
mkdir -p /backups/pixelwise
```

Backup Strategy

WHAT TO BACK UP: the database. Code is on GitHub; the model is reproducible from `train.py`. The database contains data that cannot be recreated. Every prediction, every timestamp, every confidence score.

HOW: `pg_dump` exports the database to a SQL file.

WHERE: off-site. A backup on the same server dies with the server. The `dev` VM is the natural off-site target: `pg_dump` creates the dump, `rsync` copies it to the dev VM over SSH. The two-machine setup from Block 1 pays off again.

```
# Backup script
pg_dump -U pixelwise pixelwise \
    > /tmp/pixelwise_$(date +%Y%m%d).sql
rsync -avz /tmp/pixelwise_$(date +%Y%m%d).sql \
    user@192.168.56.10:/backups/pixelwise/
```

`rsync` (https://rsync.samba.org/) is efficient file sync over SSH, incremental (only transfers changed bytes), fast, scriptable. The right tool for moving backup files between machines.

HOW TO VERIFY: restore from a backup at least once. An untested backup is not a backup:

```
# On the dev VM: restore into a test database
psql -U pixelwise -d pixelwise_test \
    < /backups/pixelwise/pixelwise_20260405.sql
```

Exercise: Verify the Backup

VERIFY THE BACKUP. Run the backup manually once. Then restore it into a test database on the dev VM and verify the data is intact.

Feature Flags

DEPLOYING NEW BEHAVIOUR WITHOUT DEPLOYING NEW CODE. The simplest form: an env var that controls which code path runs. Change the flag, restart the service, and the new behaviour is live. No code change, no PR, no CI pipeline.

```
# .env
MODEL_VERSION=v1

# In classifier.py
model_path = f"models/digit_classifier_{MODEL_VERSION}.pkl"
```

This is how teams roll out features gradually or switch between model versions without risk. Both `v1` and `v2` model files are available. Switching the model is a config change plus service restart. Block 9 will use this to compare model performance.

Exercise: Feature Flag for Model Version

ADD A FEATURE FLAG FOR MODEL VERSION. Add `MODEL_VERSION=v1` to `.env` and `.env.example`. Update `classifier.py` to load the model path dynamically. Place both `v1.pkl` and `v2.pkl` in `models/`. Switch the flag to `v2`, restart the service, and verify the model changed:

```
curl https://192.168.56.11/api/health -k
# Should show "model_version": "v2"
```

Optional: Webhooks

WEBHOOKS ARE HTTP ENDPOINTS TRIGGERED BY EXTERNAL EVENTS. The internet is event-driven. GitHub can call your server when code is pushed; Discord can receive a message when your deploy succeeds. The pattern is always the same: an event happens → an HTTP POST is sent to a URL you control.

Exercise: Trigger the Pipeline

COMMIT AND TRIGGER THE PIPELINE. Push everything you have built in this session and watch the automation run end to end:

```
git add .github/ scripts/ tests/ .env.example
git commit -m "Add CI/CD pipeline, backup cron, feature flag"
git push
```

Watch the CI workflow on GitHub. If green, merge. The deploy workflow should update the server automatically.

TAKE A SNAPSHOT of both VMs. Name them `B8-complete`.

A NOTE ON WHAT COMES NEXT. The pipeline now runs without you. Every push is linted and tested, every merge deploys, every night the database is dumped off-site. The next chapter shifts from automating the system to observing it, collecting metrics, reading logs, and noticing when something is wrong before a user does.

Security Thread

Store all deployment credentials (SSH keys, server IP) as GitHub Actions secrets, never in workflow files. `rsync` and SSH-based deploys only, never plain protocols. Backup files contain your entire database. Protect them accordingly with file permissions and consider encryption for off-site storage.

Self-Reflection and Recap

Reflection questions:

- What is the difference between Continuous Integration and Continuous Deployment?
- Why should secrets never appear in workflow YAML files?
- Why is a backup on the same server not a real backup?
- How do feature flags let you deploy new behaviour without deploying new code?
- What happens if the CI pipeline fails? Does the deploy still happen?

Milestone: Every push to `main` automatically lints and tests. Every merge deploys to the server. Nightly database backup is running and has been verified by restoring once. Feature flag for model version switching is in place.